

Industrial Games in XR

TDOA Optimization in C++

Aarhus Universitet

Institut for elektro- og computerteknologi



Journal udarbejdet af:

Rafael Moura Belling Christiansen	Au750489	202305101
Kenny Le	Au693329	202209859
Julie Nguyen	Au716092	202202059

Underviser

Henrik Daniel Kjeldsen

May 17, 2026

Indholdsfortegnelse

1	Introduktion	2
2	Hvad er Time Difference of Arrival (TDOA) i Geometrisk Algebra	2
3	TDOA i Python	2
4	Optimering i C++	3
4.1	Implementation og system design	3
4.2	Bygning og kørsel af projektet	5
4.3	Kørsel af programmet	6
5	Konklusion	6
6	Fremtidige ideer	6
7	Referencer	6

Gruppemedlem	Primært ansvar	Sekundært ansvar
Rafael	TDOA Matematik	C++ kode
Kenny	Polyscope-visualisering + CMake	C++ kode
Julie	Python-til-C++ omskrivning	Test og validering

Tabel 1: Ansvarsfordeling mellem gruppemedlemmer

1 Introduktion

Dette projekt fokuserer på at optimere en GA demo om Time Difference of Arrival (TDOA). Demoen er skrevet i Python og skal omskrives til C++ med implementeringen af et polyscope library. Dermed vil det køre hurtigere og have et brugervenligt ui af det geometriske algebra, der kan fremvise 3D graphics. Piletasterne anvendes til at bevæge target, dog kunne styring med joystick være mere ideel. Dette kunne derfor være en passende videreudvikling på projektet for fremtidige studerende. Github repo: <https://github.com/TamLe2001/TDOA.git>

2 Hvad er Time Difference of Arrival (TDOA) i Geometrisk Algebra

TDOA betyder Time Difference of Arrival. Formålet er at kunne lokalisere et eller flere objekter ved at måle tidsforskellen mellem 2 eller flere sensorer. Given et signals respons tid (t_e) fra given objekt og objektets hastighed (c), ville Time-of-Arival (TOA) af signalet fra de 2 sensore lokaseret på afstandene R_1 og R_2 være notere som:

$$t_1 = t_e + \frac{r_1}{c}$$
$$t_2 = t_e + \frac{r_2}{c}$$

På grund af man ikke helt kan stole på sensorenes signal tid, ville der bruges differencen mellem disse 2 tider til at kunne estimere lokationen af objektet. Specifikt given distancen mellem de 2 sensore kan der antages at tids-differencen ville stemme overens med distancen mellem de 2 sensore og objektet målt.

$$\sqrt{(x_t - x_1)^2 + (y_t - y_1)^2 + (z_t - z_1)^2} - \sqrt{(x_t - x_2)^2 + (y_t - y_2)^2 + (z_t - z_2)^2} = c(t_1 - t_2)$$

Hvor $[x_t, y_t, z_t]$ er objektets lokation, $t_1 - t_2$ er TDOA målingen og $[x_1, y_1, z_1]$, $[x_2, y_2, z_2]$ er sensorenes lokation.

3 TDOA i Python

Selvom de grundlæggende TDOA-ligninger ser simple ud, er de i praksis svære at implementere direkte i et Python-program. Problemet opstår, fordi koden ikke kender selve emissionstiden (t_e), hvilket resulterer i en ligning med fire ubekendte værdier: objektets tre koordinater i rummet og selve emissionstiden. Dette kræver, at man finder skæringspunktet mellem komplekse hyperbolske flader, hvilket er matematisk og programmeringsmæssigt tungt. For at gøre beregningerne nærmere vælges det i stedet at bruge Konform Geometrisk Algebra (CGA). CGA gør det muligt at omforme den ukendte emissionstid til geometri, specifikt ved at betragte problemet som skæringer mellem kugler, hvor radier ville svare til emissionstiden.

Den dertilhørende Python-kode simulerer denne proces ved at lade kuglernes størrelse variere, indtil de skærer hinanden i det korrekte punkt. Da den præcise emissionstid stadig er en ubekendt variabel, benytter koden en "brute-force" metode, hvor et loop (`for scaleR in range(40,120,1)`) systematisk tester forskellige radier, indtil skæringspunkterne mødes. Selvom denne matematiske løsning er god til at finde objektets lokation, gør de mange multivektor-beregninger i hvert trin programmet langsomt. For at løse dette problem er programmet blevet oversat til C++, hvilket gør det muligt at udføre de mange iterationer i loopet hurtigere og forhåbenligt gør det muligt at opnå realtids-performance.

4 Optimering i C++

4.1 Implementation og system design

Overblik

C++-applikationen implementerer en TDOA-baseret metode til at beregne lydkildens position i 3D-rummet ved hjælp af otte mikrofoner placeret i en kube (fire ved $z = 0$ og fire ved $z = 10$). Systemet kombinerer multilateration med scale-factor testing for at identificere mulige positionskandidater og visualiserer resultaterne i realtid. Visualiseringen håndteres af Polyscope, brugerinteraktionen af ImGui, lineær algebra af GLM og projektets build-proces af CMake.

Centrale komponenter

Datastrukturer Applikationen er bygget op omkring tre centrale datastrukturer:

- **Vec3**: En 3D-vektor i double-præcision, der understøtter addition, subtraktion, skalarmultiplikation, normalisering samt funktioner som `length()`, `dot()` og `cross()`.
- **Simulator**: Den beregningsmæssige kerne, som indeholder:
 - `mics[]`: Array med de otte mikrofonpositioner
 - `target`: Den aktuelle målposition
 - `estimate` og `estimate2`: Kandidatpunkter fra solver-algoritmen
 - `hasPair`: Boolean der angiver, om en gyldig løsning er fundet
 - `bestScale`: Den scale-factor (0.4–1.19), der gav den bedste løsning
 - `pairGap`: Afstanden mellem de to estimates
 - `dthres`: Tærskelværdi for accept (standard 0.25)
- **AppState**: Håndterer UI-tilstand og Polyscope-objekter såsom point clouds og curve networks.

Solver-algoritmen (solveTDOA) Solveren udfører følgende fem trin:

1. **Distance Calculation**: Afstandene fra target til alle otte mikrofoner beregnes.
2. **Scale Factor Loop**: Der itereres gennem scale-faktorer fra 0.40 til 1.19. For hver skala oprettes otte sfærer med $\text{radius} = \text{distance} \times \text{scale}$.
3. **Triplet Intersection**: To uafhængige tripletter af sfærer skæres:
 - Triplet 1: mics 0, 1, 4
 - Triplet 2: mics 4, 5, 2Hver triplet giver to kandidatpunkter (over/under skæringsplanet).
4. **Cross-Triplet Pairing**: Alle seks kombinationer mellem triplet-kandidater testes. Første par med afstand $< \text{dthres}$ accepteres.
5. **Return**: Det accepterede par og scale-faktoren gemmes. Hvis ingen par opfylder kriteriet, sættes `hasPair = false`.

Brugen af to uafhængige tripletter fungerer som en valideringsmekanisme: hvis begge tripletter peger på samme område, øges løsningens pålidelighed.

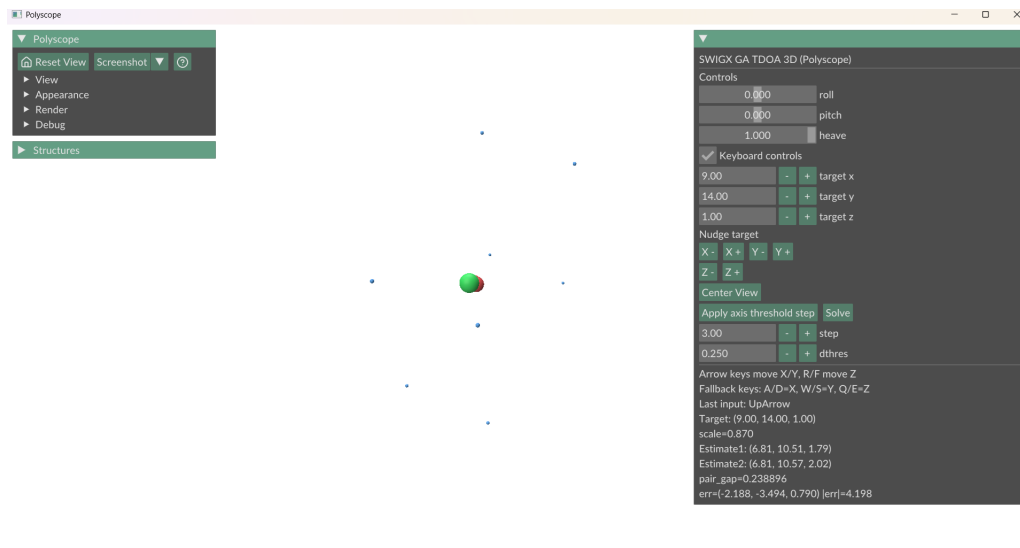
Visualiseringssystem

Fire Polyscope-objekter anvendes til at repræsentere scenen:

- **Mics (Blue spheres):** Mikrofonernes faste positioner
- **Target (Green sphere):** Den faktiske lydkilde, styret af brugerinput
- **Estimate 1 & 2 (Red spheres):** Kandidatløsninger fra solver-algoritmen
- **Estimate Link (Orange line):** Forbinder de to estimates; længden indikerer løsningens kvalitet

Brugerinterface

- **Tastaturstyring:** Arrow keys styrer X/Y, R/F styrer Z. Alternativt WASD + Q/E.
- **ImGui-panel:** Indeholder sliders til roll/pitch/heave, felter til direkte koordinatinput, knapper til små position-justeringer eller manuel solve, samt visning af target, scale, estimates og fejlfstand.
- **Event Flow:** Brugerinput → solver kører → visualisering opdateres → redraw.



Figur 1: TDOA Programmet

Centrale implementeringsdetaljer

- **Two-Candidate Design:** Solver returnerer to estimates; deres indbyrdes afstand bruges som kvalitetsindikator.
- **Real-Time Interactivity:** Hver ændring i target udløser en ny solve og opdateret visualisering.
- **Live Tuning:** dthres kan justeres under kørsel.
- **Robust Scale Search:** Test af ca. 100 scale-værdier gør algoritmen robust over for systematiske målefejl.

Kodestruktur

Nedenfor ses en forenklet udgave af kodelistykket, som indeholder den centrale kernelogik.

```
void Simulator::solveTDOA() {
    const auto trueD = distancesFrom(target);
    hasPair = false;

    for (int scaleR = 40; scaleR < 120; ++scaleR) {
        double scale = scaleR / 100.0; // 0.40 to 1.19

        // Create 8 spheres and intersect triplets
        auto pair123 = pointPairToEndPoints(S1, S2, S3_2);
        auto pair456 = pointPairToEndPoints(S1_2, S2_2, S3);
        if (!pair123 || !pair456) continue;

        // Check cross-triplet distances
        for (int k = 0; k < 6; ++k) {
            if (gaps[k] < dthres) {
                estimate = pairA[k];
                estimate2 = pairB[k];
                hasPair = true;
                bestScale = scale;
                return;
            }
        }
    }
}
```

- **Mainfile:** SWIGX_GA_TDOA_3D_polyscope_v2_improved.cpp indeholder hele implementeringen.
- **Build-konfiguration:** CMakeLists.txt definerer afhængigheder (Polyscope, GLM, ImGui).

4.2 Bygning og kørsel af projektet

Forudsætninger

CMake skal være installeret på brugerens system for at kunne bygge projektet, og skal tilføjes til systemets PATH for at kunne kaldes fra terminalen.

Bygningsinstruktioner

Alle kommandoer skal udføres i terminalen direkte fra projektets root mappe, (hvor CMakeLists.txt befinder sig).

Konfigurer build-systemet

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
```

Denne kommando:

- Opretter en ny mappe kaldet build
- Konfigurerer alle build-filer baseret på CMakeLists.txt
- Vælger Release-konfiguration (optimeret kode uden debug-info)

Kompilér projektet

```
cmake --build build --config Release
```

Denne kommando:

- Kompilerer alle sourcefiles i projektet
- Linker biblioteker og afhængigheder
- Producerer den færdige eksekverbare fil

4.3 Kørsel af programmet

Efter at programmet har kørt succesfuldt findes den eksekverbare fil i undermappen:

```
build/Release/SWIGX_GA_TDOA_3D_polyscope.exe
```

Programmet startes ved at navigere til root mappe og køre:

```
.\build\Release\SWIGX_GA_TDOA_3D_polyscope.exe
```

Eller ved at navigere direkte til Release-mappen:

```
cd build\Release
```

```
.\SWIGX_GA_TDOA_3D_polyscope.exe
```

5 Konklusion

Dette projekt optimerede GA-demoen for TDOA ved at omdanne den oprindelige Python-implementering, som var præget af langsom og gentagen logik, til en hurtigere og mere effektiv løsning i C++. Den oprindelige Python-kode brugte mange tunge beregninger og gentagne geometriske operationer, hvilket gjorde den uegnet til realtidsbrug. I den optimerede version er beregningerne flyttet til C++, hvor de kan udføres langt hurtigere, og samtidig er der udviklet et visuelt og interaktivt UI baseret på Polyscope-biblioteket, som gør det lettere at analysere og forstå TDOA-resultaterne i praksis.

6 Fremtidige ideer

Til en fremtidig implementering, kunne en ide være at implementere et joystick til at styre target. Dette kunne være en ideel implementering, da PolyScope biblioteket omdanner grafikken om til 3D. Dermed ville det optimere brugeroplevelsen fra styren med piletasterne om til et joystick.

7 Referencer

TDOA explained:

<https://se.mathworks.com/help/fusion/ug/object-tracking-using-time-difference-of-arrival.html>

PolyScope library:

<https://polyscope.run/>